

Part B §1 — Micro-Frontend Integration Architecture

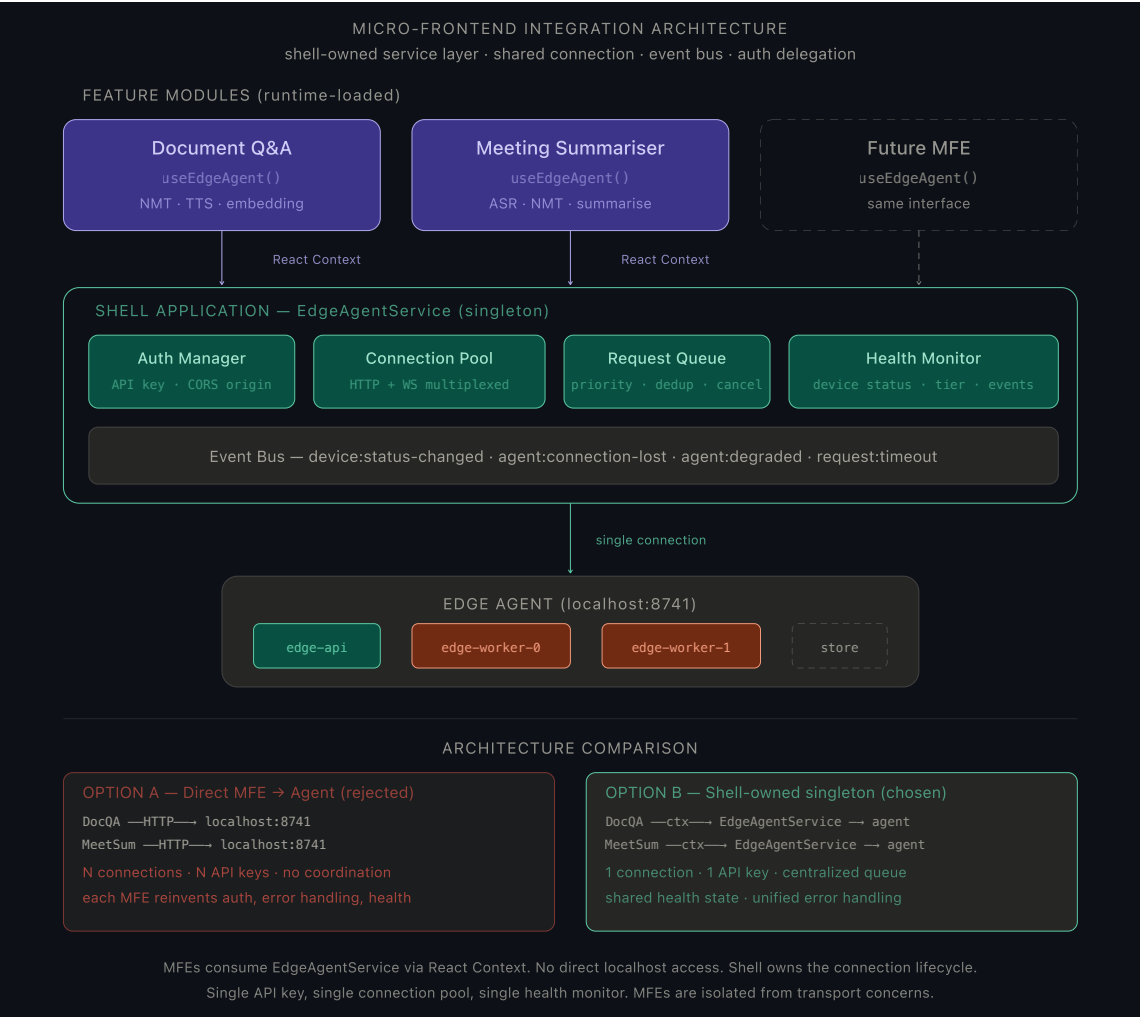
Scenario: An enterprise web application uses a Micro-Frontend (MFE) architecture. A React shell application dynamically loads feature modules at runtime. Two MFEs — **Document Q&A** and **Meeting Summariser** — need access to the Edge Agent running on `localhost:8741`.

Overview

The core question: should each MFE talk directly to the Edge Agent, or should the shell own the connection?

Answer: The shell owns the connection. The shell application provides a shared `EdgeAgentService` singleton, exposed to MFEs via React Context. MFEs never make HTTP calls to `localhost:8741` directly.

This is the right choice because the Edge Agent is a **single-process, bounded-queue, single-API-key** local service. Treating it like a cloud API that any module can hit independently leads to resource contention, duplicated auth logic, and uncoordinated failure handling.



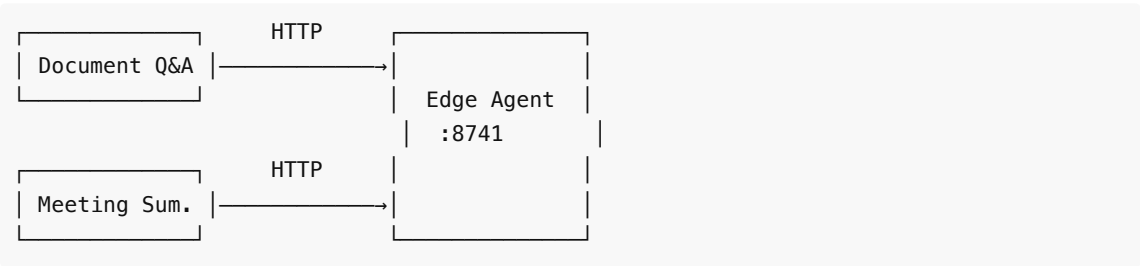
Assumptions

- 1. **Edge Agent is pre-installed** on the user's machine and binds to `127.0.0.1:8741` . It is a native desktop application, not a cloud service. The web app assumes it is present but handles its absence gracefully.
- 2. **MFEs are loaded dynamically** via Module Federation (webpack 5) or a similar runtime module system. They share the shell's React Context tree and can consume providers registered by the shell.
- 3. **Single browser tab** is the primary consumer. The architecture supports multiple tabs (each gets its own `EdgeAgentService` instance), but the 8-slot client-side queue is per-tab, not global.
- 4. **Enterprise deployment model** — the API key is provisioned per enterprise and stored in the OS keychain. The shell retrieves it once at startup. Key rotation is handled by the enterprise admin console, not by individual MFEs.
- 5. **Browser environment** — standard Chrome, Edge, or Safari. No browser extensions, Service Workers, or special permissions are required for the integration to work. The shell communicates over plain HTTP/WebSocket to localhost.
- 6. **MFEs are untrusted** relative to the shell — they are developed by different teams, may have different dependency trees, and could be compromised via supply-chain attacks. The shell treats them as consumers, not peers.

1. The Decision: Shell-Owned vs Direct MFE Access

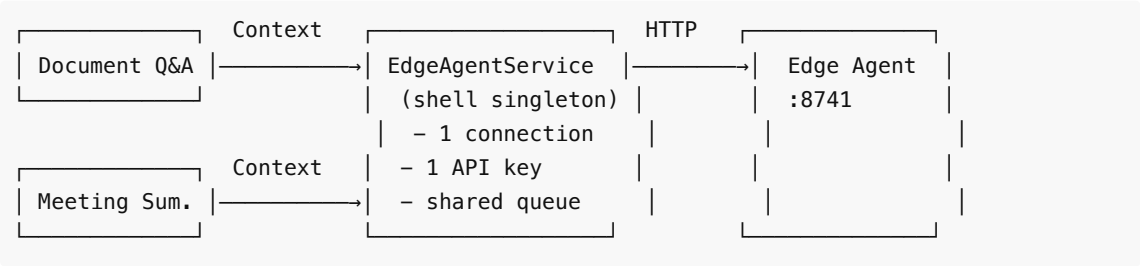
Option A — Direct MFE → Agent (rejected)

Each MFE independently creates an HTTP client, manages its own API key, opens its own WebSocket, and handles connection failures.



Option B — Shell-Owned Singleton (chosen)

The shell creates a single `EdgeAgentService` instance. MFEs consume it through a React Context provider. They call `useEdgeAgent()` and get back a typed API — no HTTP, no auth, no connection management.



2. Tradeoff Analysis

Concern	Option A (Direct)	Option B (Shell-Owned)
---------	-------------------	------------------------

Connection count	N connections (one per MFE). Each opens HTTP + potentially a WebSocket. With 5 MFEs, that's 10 sockets to a localhost service designed for 1-2 clients.	1 HTTP connection + 1 WebSocket. Multiplexed across all MFEs via request IDs.
API key management	Each MFE must separately access the API key. Key is duplicated in N memory locations. Any MFE with a dependency vulnerability can leak it.	One key, held by the shell service. MFEs never see or handle the key. Reduced attack surface.
Auth duplication	Each MFE implements its own Authorization: Bearer header injection, CORS handling, and key refresh logic. N implementations of the same thing.	One implementation in the shell. MFEs call <code>inference()</code> — auth is invisible.
Request coordination	No coordination. If both MFEs fire requests simultaneously, they compete for the Edge Agent's 64-slot queue without knowing about each other. One MFE can starve the other.	The shell's request queue applies fair-share scheduling. Per-MFE quotas prevent starvation. Priority can be assigned based on MFE type (e.g., real-time meeting transcription > batch document processing).
Health monitoring	Each MFE polls <code>/v1/health/devices</code> independently. N health-check intervals hitting the same endpoint. Each MFE parses <code>X-Edge-*</code> headers and maintains its own device state.	One health monitor in the shell. Device state is cached and shared. MFEs subscribe to events (<code>device:degraded</code> , <code>device:restored</code>) and update their UI.
Failure handling	Each MFE implements its own retry logic, reconnection, timeout handling, and degraded-mode UI. Inconsistent behavior across MFEs.	Centralized retry, reconnection, and error normalization. MFEs receive typed results (<code>{ status: 'error', code, retrying }</code>) and only implement UI-level degradation.
Bundle size	Each MFE bundles its own HTTP client, auth logic, health poller, retry logic. Duplicated code across feature modules.	HTTP client, auth, health, retry are in the shell — loaded once. MFEs are lighter.
MFE independence	MFEs are fully independent — they don't depend on the shell for agent access. Can be developed and tested in isolation against the raw API.	MFEs depend on the shell's <code>EdgeAgentProvider</code> context. Must mock the context in standalone development.
Testing	Integration tests must hit a real or mocked Edge Agent. Each MFE's test suite needs agent setup.	MFEs test against a mock <code>EdgeAgentService</code> . Only the shell's integration tests hit the real agent. Simpler test pyramid.

Why Option B wins

The deciding factor is **resource coordination**. The Edge Agent has a bounded queue (64 slots), a single API key, and communicates device degradation via response headers. These are shared resources that require coordination. Without a central coordinator:

- MFEs can't prioritize each other's requests
- MFEs can't share health state (each discovers degradation independently)
- MFEs can't prevent each other from saturating the queue
- MFEs duplicate auth, retry, and connection management

The tradeoff — reduced MFE independence — is minor. The `useEdgeAgent()` hook has a small, stable interface. Mocking it in standalone MFE development is trivial.

3. Architecture: EdgeAgentService

Service structure

```
class EdgeAgentService {
  private connection: EdgeConnection;
  private authManager: AuthManager;
  private requestQueue: FairShareQueue;
  private healthMonitor: HealthMonitor;
  private eventBus: EventEmitter;

  async inference(request: InferenceRequest): Promise<InferenceResult>;
  async stream(request: InferenceRequest): AsyncGenerator<StreamChunk>;

  subscribe(event: EdgeEvent, handler: EventHandler): Unsubscribe;
  getDeviceStatus(): DeviceStatus;
  getConnectionState(): ConnectionState;
}
```

React integration

The shell application wraps the entire MFE host in a provider:

```
// Shell application root
function App() {
  return (
    <EdgeAgentProvider config={{ endpoint: 'http://127.0.0.1:8741' }}>
      <MicroFrontendHost />
    </EdgeAgentProvider>
  );
}
```

MFEs consume the service through a hook:

```
// Inside Document Q&A MFE
function DocumentQA() {
  const { inference, stream, deviceStatus } = useEdgeAgent();
}
```

```

const handleTranslate = async (text: string) => {
  const result = await inference({
    model: 'indic-nmt-v2',
    input: text,
    params: { source_lang: 'en', target_lang: 'hi' },
    priority: 'normal',
    caller: 'doc-qa',
  });
  // result.data contains the translation
  // result.meta contains { compute, tier, latency_ms }
};

return (
  <div>
    {deviceStatus === 'degraded' && <DegradedBanner />}
    {/* ... */}
  </div>
);
}

```

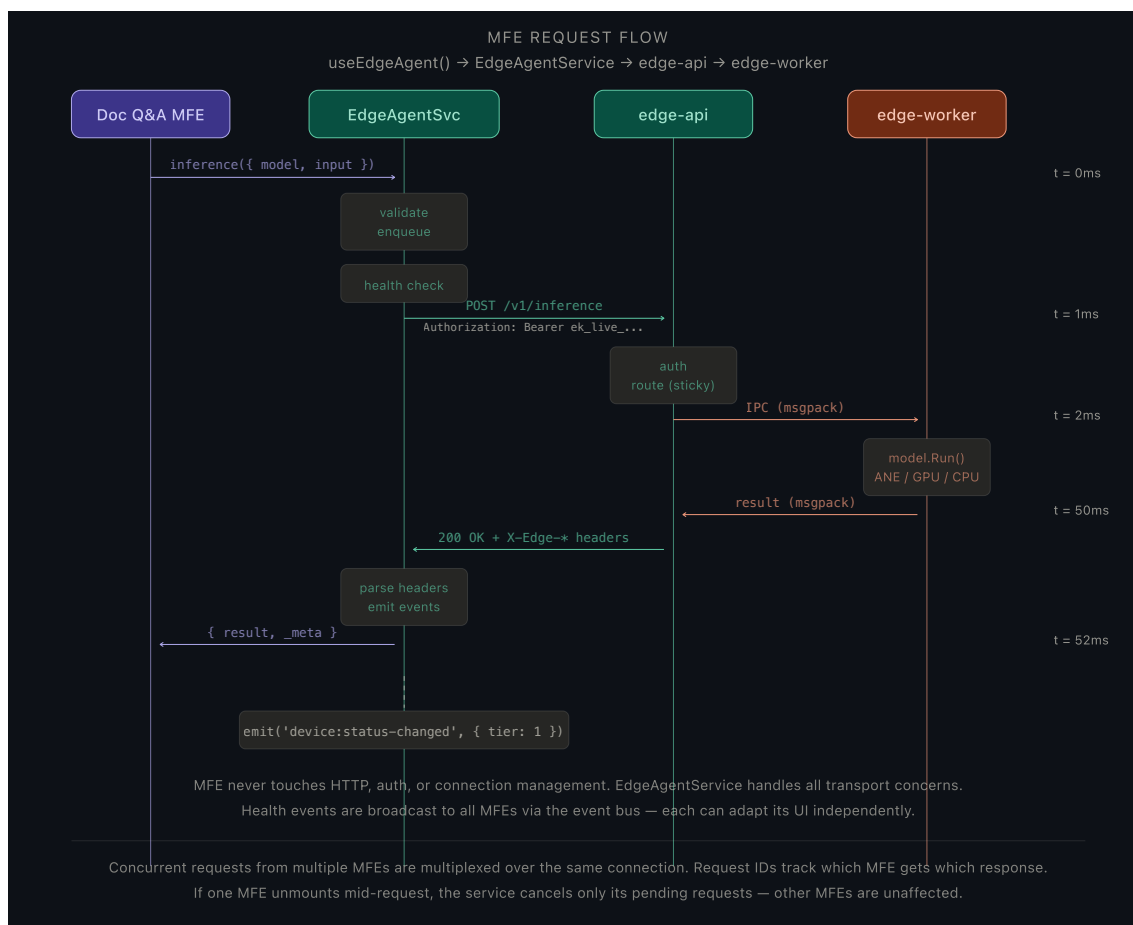
```

// Inside Meeting Summariser MFE
function MeetingSummariser() {
  const { stream, subscribe } = useEdgeAgent();

  useEffect(() => {
    const unsub = subscribe('device:status-changed', (status) => {
      if (status.tier >= 2) showSlowWarning();
    });
    return unsub;
  }, []);

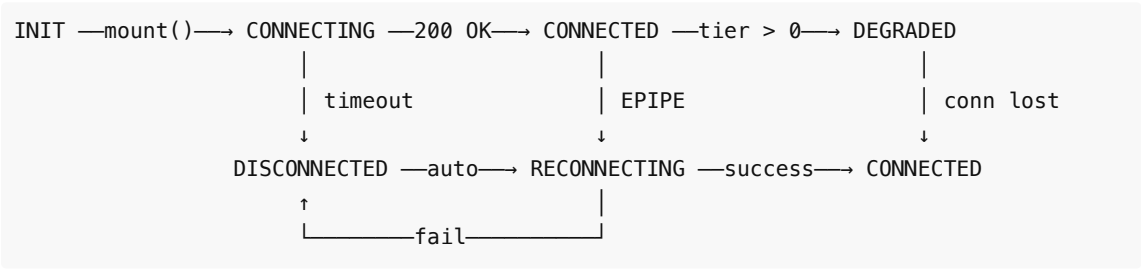
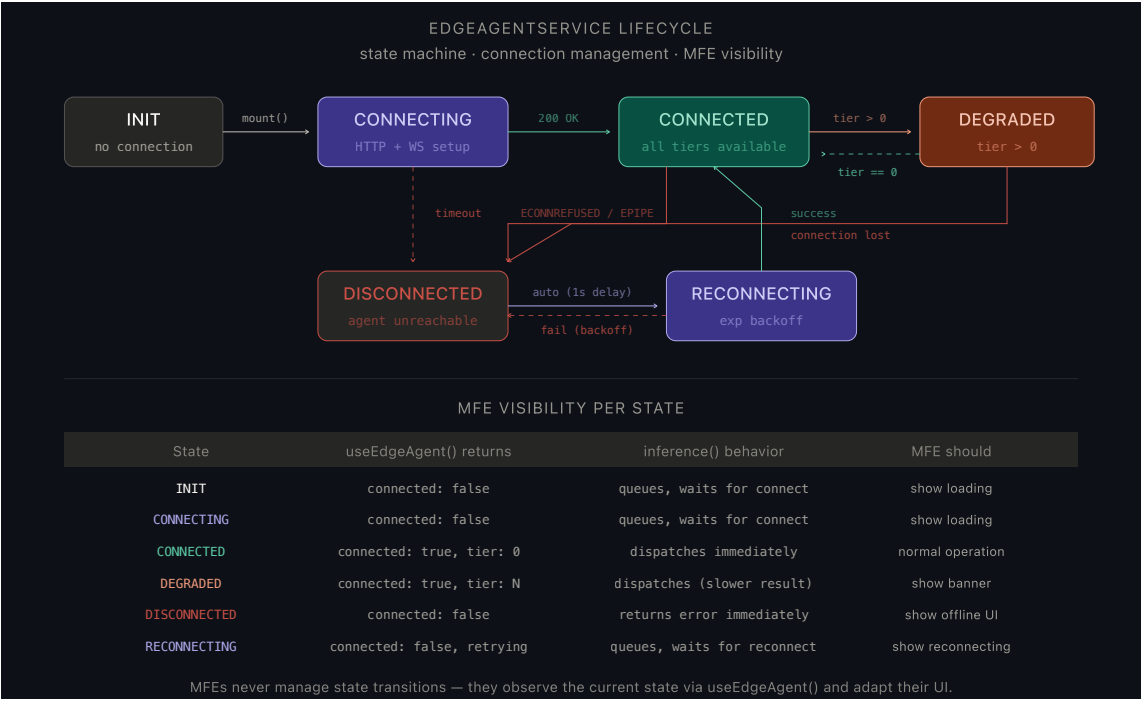
  const handleTranscribe = async function* (audioChunks:
AsyncIterable<Uint8Array>) {
    for await (const chunk of stream({
      model: 'indic-asr-v1',
      input: chunk,
      priority: 'high', // real-time transcription
      caller: 'meeting-sum',
    })) {
      yield chunk.text;
    }
  };
}

```



4. Process Lifecycle: EdgeAgentService State Machine

The `EdgeAgentService` has six states. Transitions are driven by connection events, not by MFE actions — MFEs observe the current state but never trigger transitions directly.



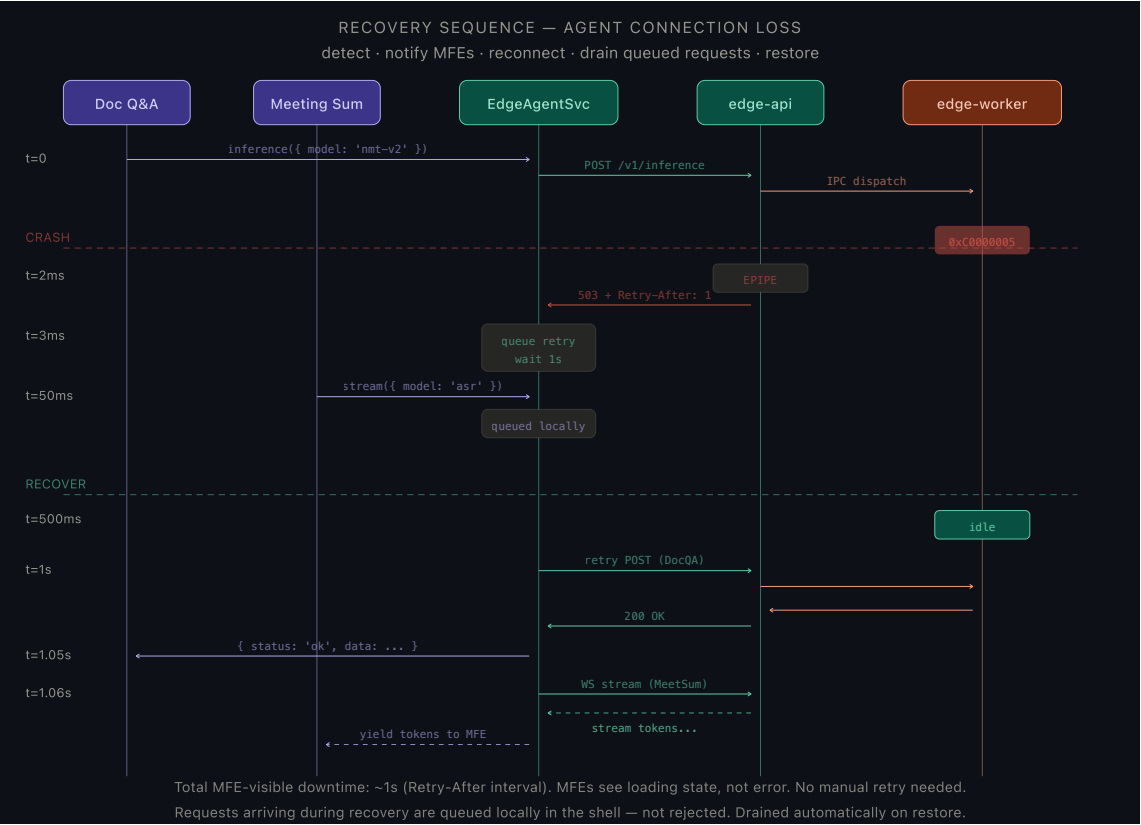
State	useEdgeAgent() returns	inference() behavior	MFE should
INIT	{ connected: false, state: 'init' }	Queues request, waits for connection	Show loading spinner
CONNECTING	{ connected: false, state: 'connecting' }	Queues request, waits for connection	Show loading spinner
CONNECTED	{ connected: true, tier: 0, status: 'healthy' }	Dispatches immediately	Normal operation
DEGRADED	{ connected: true, tier: N, status: 'degraded' }	Dispatches (result is slower)	Show "reduced performance" banner
DISCONNECTED	{ connected: false, state: 'disconnected' }	Returns error immediately	Show offline UI with retry button
RECONNECTING	{ connected: false, state: 'reconnecting', attempt: N }	Queues request, drains on restore	Show "reconnecting..." indicator

Lifecycle rules

- **On shell mount:** service transitions `INIT` → `CONNECTING` and attempts HTTP + WebSocket handshake to `127.0.0.1:8741` .
- **On successful connection:** transitions to `CONNECTED` . Drains any requests that were queued during `INIT` / `CONNECTING` .
- **On `X-Edge-Tier > 0` in any response:** transitions to `DEGRADED` . Emits `device:status-changed` event. Returns to `CONNECTED` when tier drops back to 0.
- **On connection loss** (`ECONNREFUSED` , `EPIPE` , `WebSocket close`): transitions to `DISCONNECTED` , then immediately to `RECONNECTING` .
- **Reconnect backoff:** 1s, 2s, 4s, 8s (cap). On success → `CONNECTED` . On failure → back to `DISCONNECTED` with next backoff interval.
- **On shell unmount:** service destroys all connections, cancels all in-flight requests, clears state. No persistence needed — service is stateless.

5. Recovery Sequence: Agent Connection Loss

When the Edge Agent goes down (supervisor crash, agent update, system restart), the shell service manages the entire recovery. MFEs are shielded from the complexity.



Timeline

Time	Event	Shell action	MFE experience
t = 0ms	Doc Q&A calls <code>inference()</code>	Dispatches to agent over HTTP	Sees loading state

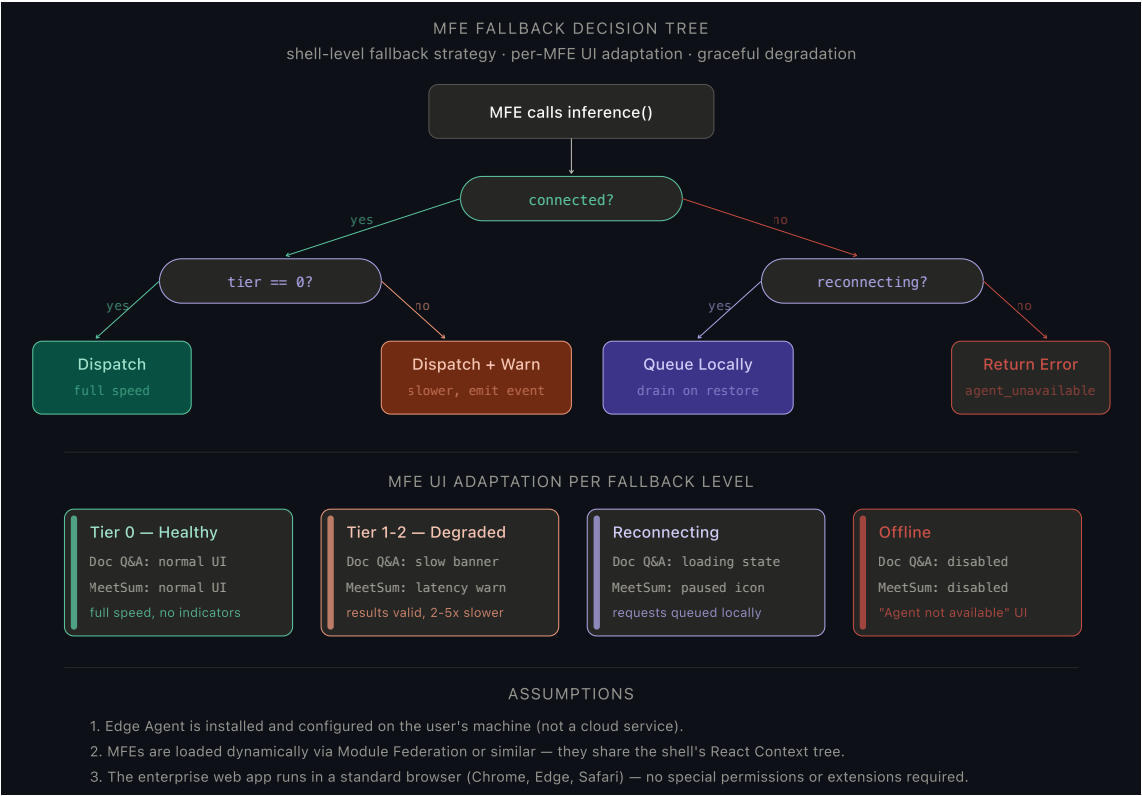
t = 2ms	Agent process dies	HTTP returns ECONNREFUSED or EPIPE	—
t = 3ms	Shell detects connection loss	Transitions to DISCONNECTED. Emits connection:lost. Starts reconnect timer (1s).	useEdgeAgent() returns { connected: false }. Doc Q&A request is queued for retry.
t = 50ms	Meeting Sum calls stream()	Request queued locally (agent is down)	Meeting Sum sees { connected: false, state: 'reconnecting' }
t = 500ms	Agent worker restarts (warm restart)	—	—
t = 1s	Reconnect timer fires	Shell attempts HTTP to agent. Succeeds. Transitions to CONNECTED. Emits connection:restored.	Both MFEs see { connected: true }
t = 1.01s	Shell drains queue	Retries Doc Q&A request. Dispatches Meeting Sum stream.	Doc Q&A receives result (~1s total delay). Meeting Sum stream begins.

Key property: zero-MFE-effort recovery

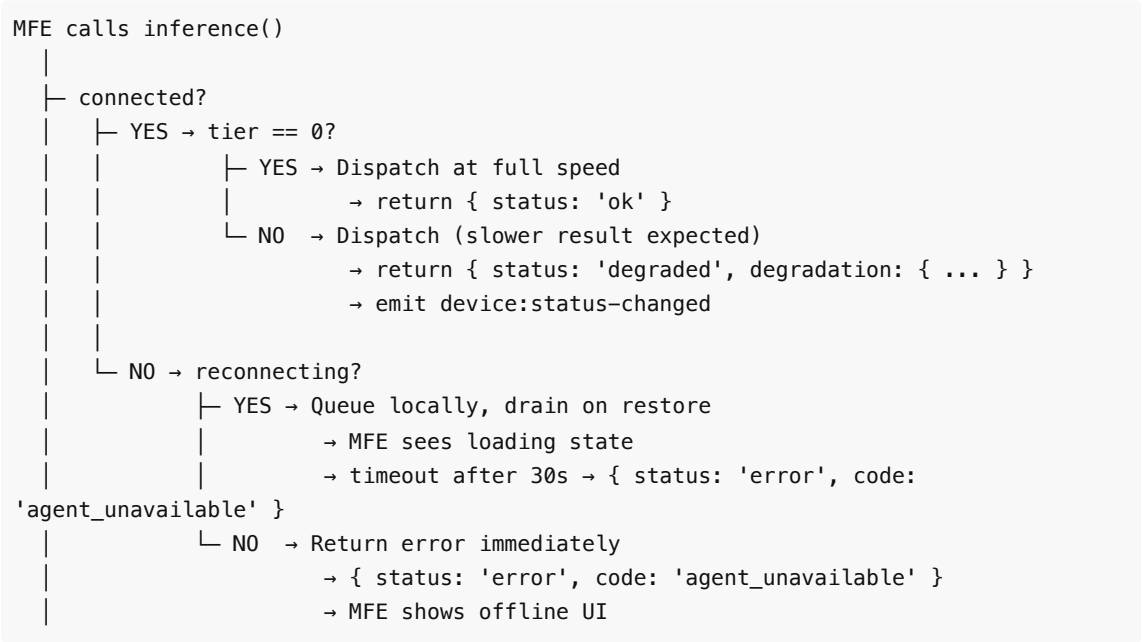
- MFEs don't implement retry logic for connection loss — the shell queues and retries automatically
- MFEs don't need to re-invoke `inference()` — the original promise resolves once the agent is back
- Both MFEs are notified simultaneously via the event bus — consistent UI state
- Requests arriving during the outage are **queued, not rejected** (unless the agent has been down for > 30s, at which point queued requests are failed with `{ status: 'error', code: 'agent_unavailable' }`)

6. Fallback Decision Tree

When an MFE calls `inference()`, the shell runs a synchronous decision tree before dispatching:



Decision flow



MFE UI adaptation per fallback level

Level	Trigger	Doc Q&A UI	Meeting Summariser UI
-------	---------	------------	-----------------------

Healthy (tier 0)	Normal operation	Normal — instant results	Normal — real-time transcription
Degraded (tier 1-2)	device:status-changed event	"Slower than usual" banner. Extends timeout.	"Transcription may be delayed" warning. Still functional.
Reconnecting	connection:lost event	Loading spinner. "Connecting to AI engine..."	Paused icon. Buffered audio queued.
Offline	Reconnect failures exceed 30s	Disabled input. "Edge Agent not available. Please ensure it's running."	Disabled recording. Same message.

The MFE's UI adaptation is optional — a basic MFE that ignores all events still works correctly. It just won't show degradation indicators.

7. Security Implications

API key isolation

Aspect	Direct (Option A)	Shell-Owned (Option B)
Key exposure	Each MFE has the key in its closure/module scope. A compromised MFE leaks the key.	Only the shell has the key. MFEs never see it. A compromised MFE can call <code>inference()</code> but can't exfiltrate the raw key.
Key rotation	Each MFE must handle key refresh independently. Race conditions during rotation.	Shell handles rotation once. MFEs are unaware of key changes.
CORS origin	Each MFE must be on the CORS allowlist. N origins to manage.	One origin (the shell's). Simpler CORS config.

Attack surface reduction

The Edge Agent listens on `127.0.0.1:8741`. In Option A, every MFE bundle contains code that knows this endpoint. If an MFE is compromised (XSS, supply-chain attack on a dependency), the attacker gets:

- The API key
- The endpoint address
- Direct access to make arbitrary inference calls

In Option B, a compromised MFE can only call `useEdgeAgent().inference()` — the shell's service layer validates the request schema before forwarding. The attacker can make inference calls (they're in the MFE, so they can call the hook), but they **cannot**:

- Bypass request validation
- Exceed the per-MFE quota
- Access the raw API key
- Call arbitrary endpoints on the agent (e.g., `/v1/health/devices` is only used by the shell's health monitor)

Content Security Policy

Option B enables a stricter CSP:

```
Content-Security-Policy: connect-src 'self';
```

MFEs are not allowed to make network requests at all — only the shell's service worker / main thread can talk to `localhost:8741` . With Option A, the CSP must allow `connect-src http://127.0.0.1:8741` for every MFE origin.

8. Resource Coordination

The problem: bounded queue contention

The Edge Agent's gateway has a 64-slot bounded queue (16 high-priority + 48 normal). Without coordination, two MFEs can easily saturate it:

- Meeting Summariser streams real-time ASR (high-priority, continuous)
- Document Q&A fires batch translations (normal-priority, bursty)

If Meeting Summariser fills the high-priority lane, Document Q&A's requests back up. If Document Q&A fires 50 translation requests at once, it triggers `429 Too Many Requests` — not just for itself, but for Meeting Summariser too.

The solution: Fair-share queue in the shell

The `EdgeAgentService` maintains a client-side request queue with per-MFE quotas:

```
interface FairShareConfig {
  maxConcurrent: number; // total in-flight requests (default: 8)
  perCallerMax: number; // max in-flight per MFE (default: 4)
  priorityMapping: Record<string, 'high' | 'normal'>;
}
```

MFE	Default quota	Priority	Rationale
Meeting Summariser	4 concurrent	high	Real-time — latency-sensitive
Document Q&A	4 concurrent	normal	Batch — throughput-sensitive
Total in-flight	8	—	Leaves headroom in the agent's 64-slot queue for retries

The shell queue enforces these limits **before** hitting the agent. This means:

- MFEs never see `429` errors — the shell queues excess requests locally
- Priority is enforced at the shell level, not discovered by trial-and-error at the agent level
- The agent's queue is never saturated by one MFE

Connection pooling

```

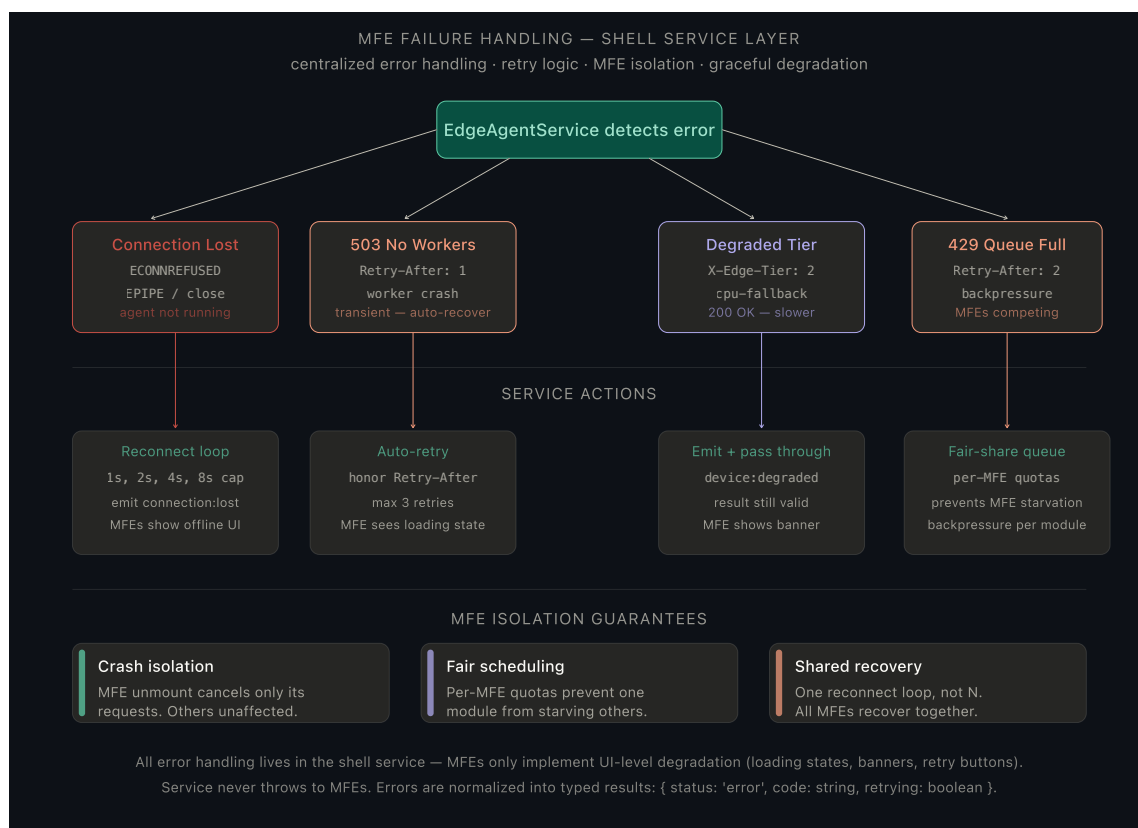
class EdgeConnection {
  private httpClient: HttpClient; // single HTTP/1.1 keep-alive connection
  private wsClient: WebSocket; // single WebSocket for streaming
  private inflightRequests: Map<string, PendingRequest>;

  async request(req: AgentRequest): Promise<AgentResponse> {
    const requestId = generateId();
    this.inflightRequests.set(requestId, { resolve, reject, caller: req.caller
  });
    // ... send over shared connection
  }
}

```

One HTTP connection, one WebSocket. Requests are multiplexed via `request_id` . When a response arrives, the connection routes it to the correct MFE's pending promise based on the `request_id` .

9. Failure Handling



Failure taxonomy

Failure	HTTP code	Service action	MFE experience
Agent not running	ECONNREFUSED	Reconnect loop (1s, 2s, 4s, 8s cap). Emit	useEdgeAgent() returns { connected: false }. MFE shows offline UI.

		connection:lost.	
Worker crash	503 + Retry-After: 1	Auto-retry after Retry-After seconds. Max 3 retries.	MFE sees loading state. If all retries fail, receives { status: 'error', code: 'no_workers' }.
Queue full	429 + Retry-After: 2	Shell reduces its own dispatch rate. Queues excess locally.	MFE sees longer loading time, not an error. Shell absorbs backpressure.
Degraded tier	200 + X-Edge-Tier: 2	Parse headers, update cached device status, emit device:degraded.	MFE receives a valid result (slower). Can show a "reduced performance" banner if it subscribes to the event.
Request timeout	—	Cancel in-flight request. Return timeout error.	MFE receives { status: 'error', code: 'timeout' }. Can retry or show fallback UI.
MFE unmount	—	Cancel all pending requests for that MFE's caller ID.	Other MFEs unaffected. No orphaned requests.

Key property: MFE isolation during failures

When Meeting Summariser crashes or unmounts, the shell cancels its pending requests (identified by `caller: 'meeting-sum'`). Document Q&A's requests are completely unaffected — they continue to resolve normally.

When the agent connection drops, **one** reconnect loop runs (in the shell), not two (one per MFE). All MFEs are notified via the event bus simultaneously.

Error normalization

The service **never throws** to MFEs. Every call returns a typed result:

```
type InferenceResult =
  | { status: 'ok'; data: unknown; meta: ResponseMeta }
  | { status: 'error'; code: ErrorCode; message: string; retrying: boolean }
  | { status: 'degraded'; data: unknown; meta: ResponseMeta; degradation: DegradationInfo };
```

MFEs switch on `status` and render accordingly. They never parse HTTP status codes, headers, or connection errors.

10. Scheduler Architecture & Queueing Model

The shell's `FairShareQueue` sits between MFEs and the Edge Agent's own bounded queue:

```
MFEs → FairShareQueue (shell, 8 slots) → EdgeConnection → edge-api queue (64 slots) → workers
```

Two-level queueing

Level	Location	Size	Purpose
L1 (shell)	FairShareQueue in browser	8 in-flight	Per-MFE fairness, priority, client-side backpressure
L2 (agent)	edge-api gateway queue	64 slots	Server-side scheduling, sticky routing, worker dispatch

The shell never sends more than 8 concurrent requests to the agent. This means the agent's 64-slot queue is never saturated by this browser tab, leaving room for other clients (Electron apps, other browser tabs, extensions).

Priority mapping

The shell maps MFE `caller` IDs to priority levels:

```
const priorityMap = {
  'meeting-sum': 'high',    // real-time transcription
  'doc-qa': 'normal',      // batch translation
  'default': 'normal',
};
```

High-priority requests skip ahead in the shell queue. The agent's gateway independently classifies stream vs. batch, so the shell priority and agent priority are **aligned but independent** — the shell can't override the agent's scheduling, but it can ensure high-priority requests reach the agent first.

Backpressure propagation

When the agent returns `429` :

1. Shell pauses its dispatch loop
2. Honors `Retry-After` header
3. Reduces `maxConcurrent` from 8 to 4 temporarily
4. Queues new MFE requests locally (they see longer load times, not errors)
5. Restores `maxConcurrent` after successful requests resume

11. API Contract Examples

MFE → Shell (React Context API)

```
// Inference request (MFE calls this)
const result = await inference({
  model: 'indic-nmt-v2',
  input: 'Hello, how are you?',
  params: { source_lang: 'en', target_lang: 'hi' },
  priority: 'normal',
  caller: 'doc-qa',
});

// Successful response
{
```

```

    status: 'ok',
    data: { text: 'नमस्ते, आप कैसे हैं?' },
    meta: {
      compute: 'ane',
      tier: 0,
      latency_ms: 42,
      device_status: 'healthy',
    }
  }

// Degraded response (still successful)
{
  status: 'degraded',
  data: { text: 'नमस्ते, आप कैसे हैं?' },
  meta: {
    compute: 'cpu-fallback',
    tier: 2,
    latency_ms: 185,
    device_status: 'critical',
  },
  degradation: {
    reason: 'ane-faulted',
    latency_factor: 4.4,
    estimated_restore_s: 28,
  }
}

// Error response
{
  status: 'error',
  code: 'no_workers',
  message: 'All inference workers are recovering. Retrying.',
  retrying: true,
}

```

Shell → Edge Agent (HTTP — hidden from MFEs)

```

POST /v1/inference HTTP/1.1
Host: 127.0.0.1:8741
Authorization: Bearer ek_live_...
Content-Type: application/json
X-Request-Id: req_7f3a9b
X-Caller-Id: doc-qa

{"model": "indic-nmt-v2", "input": "Hello, how are you?", "params": {"source_lang":
"en", "target_lang": "hi"}}

```

```

HTTP/1.1 200 OK
X-Edge-Compute: ane
X-Edge-Latency-Factor: 1.0

```


X-Edge-Device-Status: healthy

X-Edge-Tier: 0

```
{"result": {"text": "नमस्ते, आप कैसे हैं?"}, "_meta": {"compute": "ane", "tier": 0, "latency_ms": 42}}
```

WebSocket stream (shell manages, MFE gets AsyncGenerator)

```
// MFE code
for await (const chunk of stream({ model: 'indic-asr-v1', input: audioBlob, caller: 'meeting-sum' })) {
    appendTranscript(chunk.text);
}

// Under the hood, the shell manages the WebSocket frames:
// → {"type":"meta","compute":"ane","tier":0}
// → {"type":"token","text":"आज"}
// → {"type":"token","text":"की"}
// → {"type":"token","text":"बैठक"}
// → {"type":"done","total_ms":320}
```

Health status (shell polls, MFEs subscribe)

```
// MFE subscribes to events
const unsub = subscribe('device:status-changed', (status) => {
    // status: { tier: 1, compute: 'metal-gpu', latency_factor: 1.8, restore_s: 28 }
    if (status.tier >= 2) showSlowWarning();
});
```

12. State/Event Payload Examples

EdgeAgentService internal state

```
{
  "connection": {
    "state": "connected",
    "endpoint": "http://127.0.0.1:8741",
    "http_alive": true,
    "ws_alive": true,
    "reconnect_attempt": 0
  },
  "device": {
    "tier": 1,
    "compute": "metal-gpu",
    "status": "degraded",
    "latency_factor": 1.8,
    "estimated_restore_s": 28,
    "last_updated": "2025-01-15T10:24:00Z"
  }
}
```

```

    },
    "queue": {
      "in_flight": 3,
      "max_concurrent": 8,
      "per_caller": {
        "doc-qa": { "in_flight": 1, "queued": 0, "max": 4 },
        "meeting-sum": { "in_flight": 2, "queued": 0, "max": 4 }
      }
    }
  }
}

```

Event bus payloads

```

// Connection events
{ type: 'connection:lost', timestamp: '2025-01-15T10:23:45Z' }
{ type: 'connection:reconnecting', attempt: 2, next_retry_ms: 4000 }
{ type: 'connection:restored', downtime_ms: 6200 }

// Device events (parsed from X-Edge-* headers)
{ type: 'device:status-changed', tier: 2, compute: 'cpu-fallback', status:
'critical', latency_factor: 4.2 }
{ type: 'device:restored', tier: 0, compute: 'ane', downtime_s: 90 }

// Queue events
{ type: 'queue:backpressure', reduced_max: 4, reason: '429_received' }
{ type: 'queue:restored', max_concurrent: 8 }

// Per-MFE events
{ type: 'caller:request-queued', caller: 'doc-qa', position: 3, reason:
'quota_exceeded' }
{ type: 'caller:requests-cancelled', caller: 'meeting-sum', count: 2, reason:
'unmount' }

```

Request lifecycle (internal tracking)

```

{
  "request_id": "req_7f3a9b",
  "caller": "doc-qa",
  "model": "indic-nmt-v2",
  "priority": "normal",
  "state": "in_flight",
  "queued_at": "2025-01-15T10:24:00.100Z",
  "dispatched_at": "2025-01-15T10:24:00.102Z",
  "response_at": null,
  "retry_count": 0,
  "abort_controller": "<AbortController>"
}

```

13. Design Tradeoff Explanations

Decision	Chosen	Alternative	Rationale
Shell-owned singleton	Single <code>EdgeAgentService</code> in shell	Each MFE connects directly	Edge Agent has bounded queue (64) and single API key. Central coordination prevents starvation, duplicated auth, and N connections. MFE independence tradeoff is minor — mock the hook in dev.
React Context for MFE access	<code>EdgeAgentProvider</code> + <code>useEdgeAgent()</code>	Module Federation shared module / global singleton	Context is the standard React composition primitive. Follows MFE best practices. Easy to mock in tests. No global state pollution.
Fair-share queue (client-side)	Per-MFE quotas with shell-level queue	Let MFEs compete for the agent's queue	Prevents starvation. MFEs never see 429 — the shell absorbs backpressure locally. Agent's queue isn't saturated by one browser tab.
Typed result (no throws)	Return <code>{ status, data, meta }</code> union	Throw on HTTP errors	MFEs shouldn't need <code>try/catch</code> for expected failures. Typed result makes degradation a first-class UI state, not an exception.
Event bus for health	Shell emits events, MFEs subscribe	MFEs poll health endpoint independently	One poll interval, shared state. MFEs react to events instead of duplicating health checks.
Single HTTP + WS connection	Multiplexed via request IDs	Per-MFE connections	One connection to a localhost service is plenty. Multiplexing avoids N TCP handshakes and N socket file descriptors.
8-slot shell queue	Conservative limit well below 64	Match agent's 64-slot limit	Leaves headroom for other clients (Electron, extensions, other tabs). Shell shouldn't assume it's the only consumer.
Priority mapping by caller	<code>meeting-sum: high</code> , <code>doc-qa: normal</code>	All requests same priority	Real-time transcription has stricter latency requirements than batch translation. Priority at the shell ensures high-priority reaches the agent first.
Cancel on unmount	<code>AbortController</code> per MFE caller	Let requests complete even after unmount	Orphaned requests waste agent queue slots. Cancellation frees resources for active MFEs.

14. Performance & Reliability

Performance

Metric	Value	Notes
Shell service overhead	< 0.5ms per request	Context dispatch + queue check + header injection
Connection setup (once)	~5ms	HTTP keep-alive + WebSocket upgrade to localhost
Request multiplexing	~0.1ms	Request ID generation + Map lookup
Event bus dispatch	~0.05ms	Synchronous event emission to subscribed MFEs
Health poll interval	10s	Single poll, cached result shared with all MFEs
Queue fairness check	~0.01ms	Simple counter comparison per caller
Memory overhead	~50KB	Service singleton + connection state + queue state

Reliability

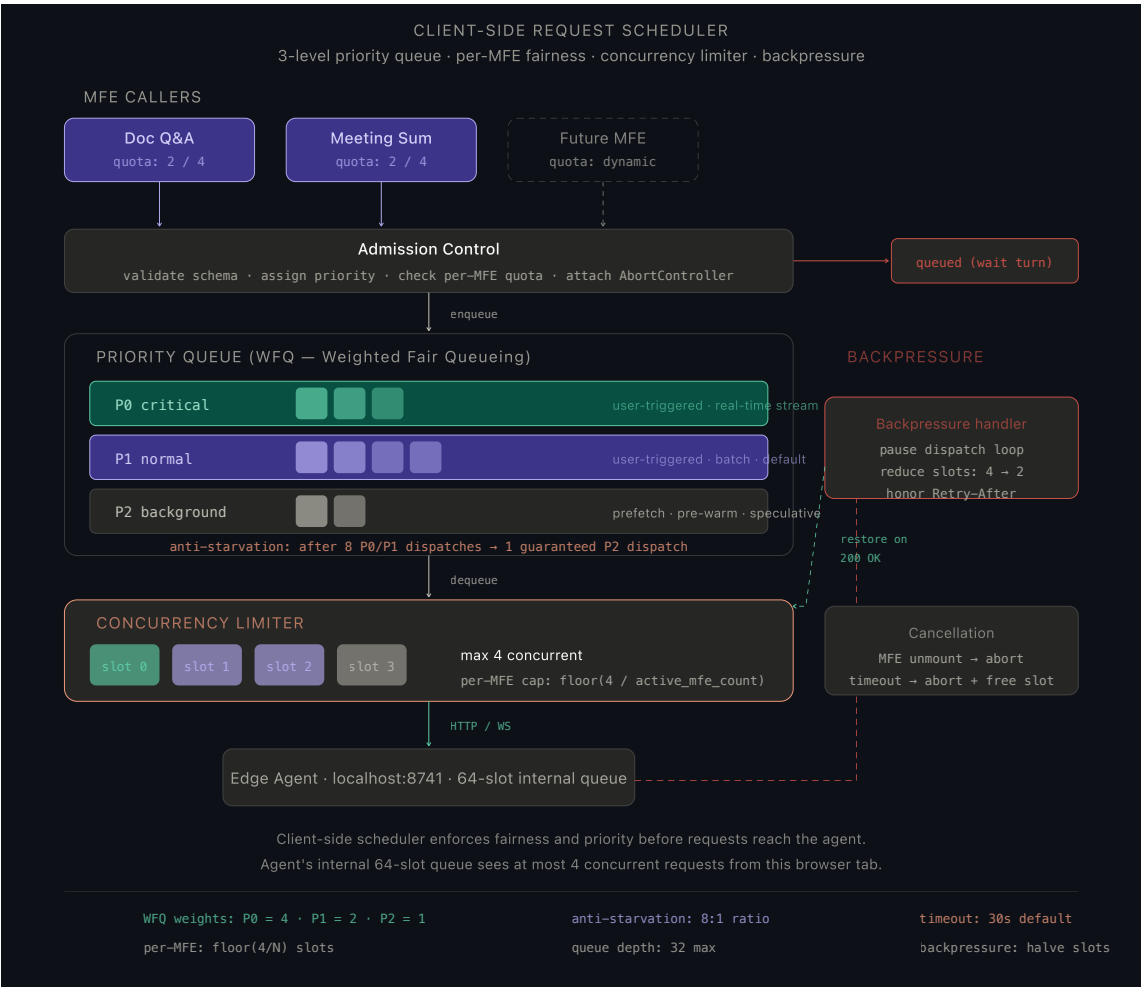
Concern	Mitigation
Shell crash	Service is stateless. Shell restart recreates service. No state to persist.
MFE crash	AbortController cancels that MFE's pending requests. Other MFEs unaffected.
Agent restart	Shell detects broken connection, enters reconnect loop (exp backoff 1s → 8s). All MFEs see <code>connected: false</code> simultaneously.
Agent queue full	Shell absorbs backpressure — reduces dispatch rate, queues locally. MFEs see longer load times, not errors.
Degraded device	Shell parses <code>X-Edge-*</code> headers once, caches state, emits event. MFEs adapt UI (banner, timeout adjustment) without polling.
MFE starvation	Per-MFE quotas (default: 4 concurrent). One MFE can't monopolize the connection.
Key rotation	Shell handles rotation. MFEs are unaware. No race condition across N modules.
WebSocket drop	Shell reconnects. Streaming MFEs receive <code>{ status: 'error', code: 'stream_interrupted' }</code> and can re-initiate.

Reliability guarantees

- **No MFE sees raw HTTP errors:** all errors are normalized into typed results
- **No MFE manages connections:** shell handles connect, reconnect, keep-alive
- **No MFE competes unfairly:** fair-share queue enforces quotas
- **No MFE is affected by another's crash:** requests are isolated by `caller` ID
- **Health state is consistent:** all MFEs see the same device status at the same time

15. Streaming + Capacity Contention Contract (Mandatory Scenario)

Scenario: Meeting Summariser has an active SSE stream and the Edge Agent is at full capacity (4/4). User opens Document Q&A and submits a query.



15.1 Expected scheduler behavior

- 1. Active streaming slots are preserved unless user cancellation occurs.
- 2. New Document Q&A request is accepted into shell queue (not rejected immediately).
- 3. Fair-share policy guarantees queue progress for both MFEs.
- 4. Priority policy applies: user-triggered requests outrank background tasks, but anti-starvation aging prevents indefinite wait.

15.2 What each MFE receives

MFE	Immediate response	Ongoing updates	Terminal state
Meeting Summariser	Stream continues	stream:progress and occasional queue:pressure hints	stream:done Or stream:cancelled
Document Q&A	queued acknowledgement with position and ETA	Queue position updates + state transitions	ok, error, or cancelled

15.3 Event/state payload examples

```
{
  "type": "caller:request-queued",
  "caller": "doc-qa",
  "request_id": "req_7f3a9b",
  "queue_position": 2,
  "estimated_start_ms": 1800,
  "priority": "high"
}
```

```
{
  "type": "stream:progress",
  "caller": "meeting-sum",
  "request_id": "req_9b2f4c",
  "tokens_emitted": 256,
  "active_slots": 4,
  "queue_depth": 3
}
```

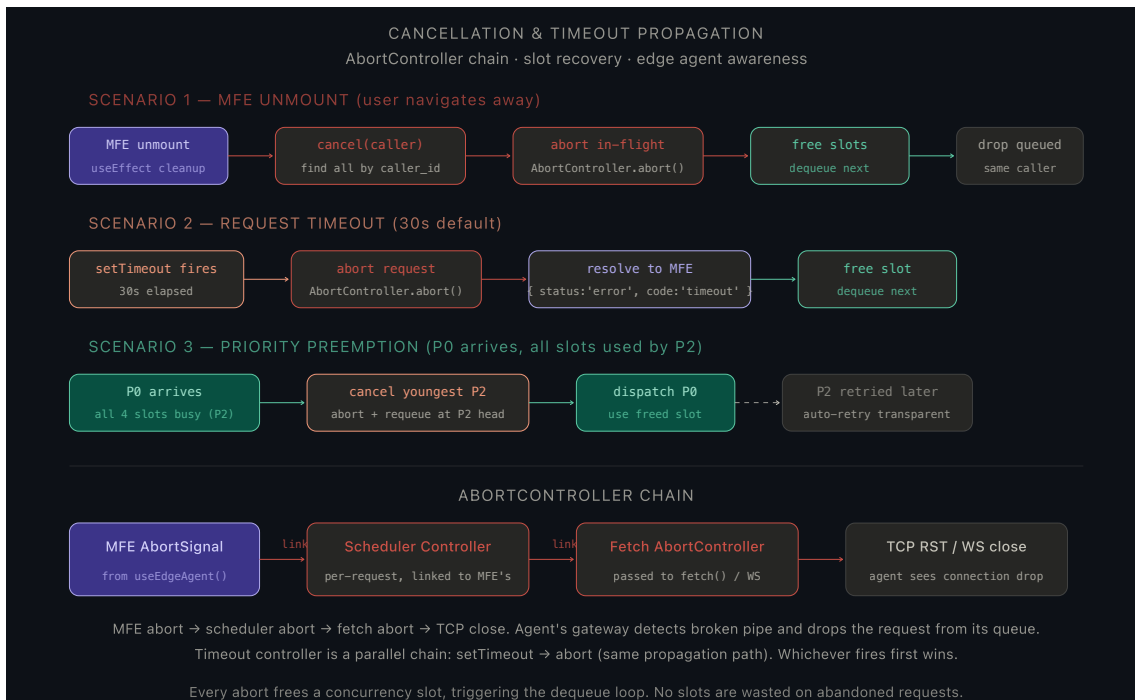
```
{
  "type": "caller:state-transition",
  "caller": "doc-qa",
  "request_id": "req_7f3a9b",
  "from": "queued",
  "to": "in_flight",
  "slot": "worker-1"
}
```

15.4 Queue visibility rules

1. Each MFE can view detailed state for its own requests.
2. Each MFE can view only aggregate queue pressure for other callers (no payload/body leakage).
3. Shell admin diagnostics can view full queue details for debugging.

15.5 Cancellation and timeout propagation

- MFE cancellation triggers `AbortController`, removes queued item if not started, or sends cancellation signal if in-flight.
- Streaming cancellation emits `stream:cancelled` and immediately frees one slot.
- Request timeout emits typed error (`code: timeout`) and advances queue.



15.6 Retry semantics

1. Transport failures use bounded exponential retry in shell (1s, 2s, 4s , max 3 attempts).
2. Server 503 / 429 honors Retry-After when present.
3. Retries reuse the same request_id for idempotency and deduplication.

This guarantees predictable UX under contention while preserving fairness and correctness.

Summary

Question	Answer
Shell or direct?	Shell owns the connection. EdgeAgentService singleton exposed via React Context. MFEs call useEdgeAgent().
Why not direct?	Bounded queue, single API key, shared health state — these are coordinated resources. N independent clients can't coordinate without a mediator.
Security	API key held only by shell. MFEs can't exfiltrate it. Stricter CSP (connect-src 'self'). Request validation at the shell layer.
Resource coordination	Fair-share queue with per-MFE quotas (4 concurrent each). Priority mapping (real-time > batch). Client-side backpressure absorption.
Failure handling	Centralized retry, reconnect, error normalization. MFEs get typed results, never raw HTTP. One reconnect loop, not N. Cancel on unmount.